

Qubit Evolution

Rudi Giuseppe, Cristiano Francesco

Artificial Intelligence and Computer Science / University of Calabria

July 27, 2026

Abstract

Quantum computing has gained strong interest in recent years, but simulating many interacting qubits is slow and expensive because the system grows very quickly with the number of qubits and its behaviour is probabilistic. In this project, we use deep learning to learn qubit time-series dynamics from short sequences and to predict future ones. The same pipeline can also fill in missing data (imputation) and is controlled by a single configuration file, so we can change preprocessing, training, and evaluation without editing the code. Our results show that learning the correlation patterns between qubits is challenging, which limits the precision of the predictions. For short forecasting horizons, the models can capture the overall trend and direction of the dynamics, but it does not match the target trajectory perfectly. As the prediction horizon increases, the performance degrades exponentially.

1 Introduction

Quantum computing is a promising approach, but studying and simulating many interacting **qubits** is difficult. The main reason is that the size of the quantum state grows exponentially, and the qubits can develop complex correlations. For this reason, **data-driven methods** are useful: they can learn temporal patterns directly from observed trajectories.

In our setting we monitor **magnetization** and **correlation** signals, where the correlation of one qubit can depend on all the others. Because these interactions are non-local, the dynamics are hard to model, so we use data-driven methods that learn temporal patterns directly from observed trajectories.

In this report, we use deep learning to model qubit time series with two prediction tasks: (i) **forecasting** future time steps from a short observation window and (ii) **super-resolution**, reconstructing missing points.

We compare two deep learning frameworks: an approach based on **LSTM** and one based on **Transformer (TRN)**. Overall, we built five seq2seq models: three LSTM forecasting models with different decoder modes, one TRN model for **forecasting**, and one TRN model for **super-resolution**. All models, except for the last one we talked about, follow an encoder - decoder structure, where an encoder maps the input sequence to a latent representation and a decoder generates the output sequence. For the forecasting models, we study three decoding strategies: **full-seq**, **step-wise**, and a **hybrid**. The term **hybrid** refers to the decoder's ability to operate in both full-seq and step-wise modes in order to adapt based on the chosen strategy.

2 Dataset and Preprocessing

2.1 Dataset

Our input dataset contains trajectories generated from the PXP model with $N = 10$ qubits. The dynamics are sampled every $\Delta t = 0.02$ s up to $T_{\text{fin}} = 20$ s, giving $T = T_{\text{fin}}/\Delta t + 1 = 1001$ time steps per trajectory (with $N_{\text{traj}} = 400$ trajectories in total). Each trajectory is stored as a matrix with T rows and $1 + N(N + 1)/2$ columns. The first column is time t , the next N columns are the magnetizations $(m_1, \dots, m_{10})$, and the remaining 45 columns are the pairwise correlations $(c_{12}, c_{13}, \dots, c_{110}, c_{23}, \dots, c_{910})$.

2.2 Preprocessing

Figure 1 summarizes the full data preparation workflow, from loading the raw trajectories to producing the final tensors that are used as inputs and targets for the different model fits. Before building the sliding windows for each split (train/validation/test), we apply standardization to the data to improve training stability and to make the different variables comparable in scale.

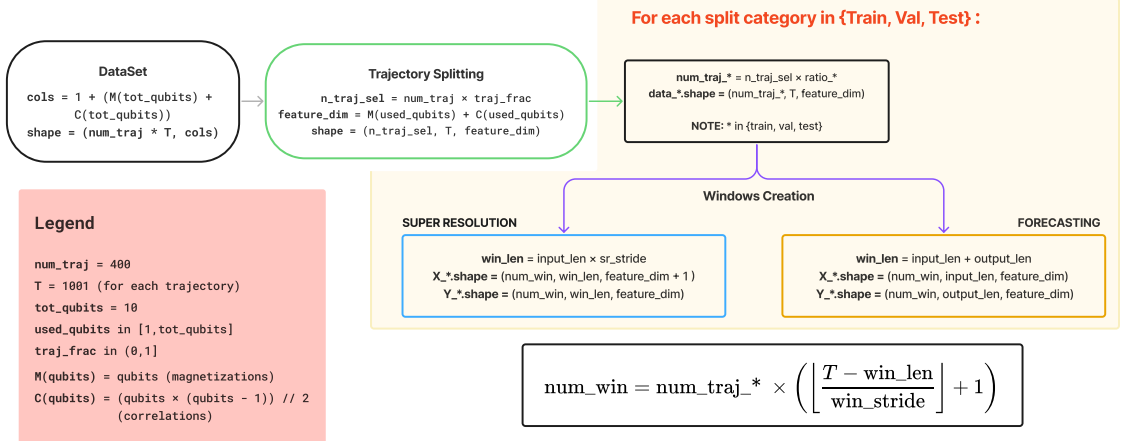


Figure 1: Preprocessing pipeline and tensor shapes across stages.

Note. In *super-resolution*, input_seq_len is the number of observed points (every sr_stride steps), while the model predicts the sequence including missing points: $\text{output_seq_len} = \text{input_seq_len} \cdot \text{sr_stride}$.

3 Models

3.1 Models overview

Figure 2 provides a temporal roadmap of the model development in this project: as the work progressed, we moved from simpler baselines to more advanced architectures and training strategies. For each model, we explain the main limitations caused by theoretical or structural constraints and from unimplemented features.

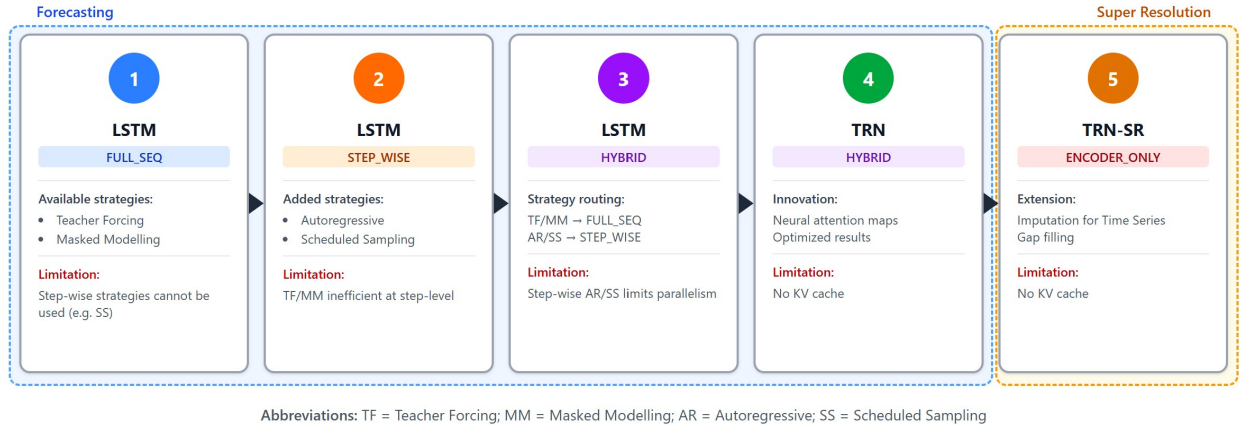


Figure 2: Roadmap of the developed models and decoder modes.

Note. Some limitations are deferred to Section 5 (Future work).

We formulate super-resolution as a simple imputation task: starting from a sequence with regularly missing time steps, the model receives the input where missing points are replaced by a fixed mask value and learns to predict only the missing values. We compute the loss on missing points (or on all points with a smaller weight on observed ones) to focus training on reconstruction quality.

3.2 Model Architectures

Notation: B = batch size, T_{in} = input length, T_{dec} denotes the decoder sequence length, which can be the full output length T_{out} or the current curriculum horizon T_{hor} , D = feature dimension, d_{lat} = latent dimension, d_{model} = model dimension, d_{ff} = feed-forward dimension, H = attention heads, W = window size, N = number of stacked blocks. All hyperparameters are configured via YAML files.

3.2.1 Hybrid LSTM

Component	Layer	Output shape
Encoder	LSTM ₁	(B, T_{in}, d_{lat})
	LSTM ₂	states h_1, c_1, h_2, c_2
Decoder	LSTM ₁	(B, T_{dec}, d_{lat})
	LSTM ₂	(B, T_{dec}, d_{lat})
	Dense + ReLU	$(B, T_{dec}, 128)$
	Dropout	$(B, T_{dec}, 128)$
	Dense	(B, T_{dec}, D)

In full-seq mode, the decoder LSTM is called once on the whole decoder input sequence. In step-wise mode, the model explicitly iterates over the output horizon and calls a one-step decoder at each timestep.

3.2.2 Hybrid Transformer

Component	Layer	Output shape
Encoder	Dense + Pos. Embedding + Dropout	(B, T_{in}, d_{model})
	$\times N$ LayerNorm + MHA (unmasked) + Residual	(B, T_{in}, d_{model})
	$\times N$ LayerNorm + FFN (d_{ff}) + Residual	(B, T_{in}, d_{model})
	Memory M (not a layer)	(B, T_{in}, d_{model})
Decoder	Dense + Pos. Embedding + Dropout	(B, T_{dec}, d_{model})
	$\times N$ LayerNorm + Causal MHA + Residual	(B, T_{dec}, d_{model})
	$\times N$ LayerNorm + Cross-Attn (K, V from M) + Residual	(B, T_{dec}, d_{model})
	$\times N$ LayerNorm + FFN (d_{ff}) + Residual	(B, T_{dec}, d_{model})
	Dense	(B, T_{dec}, D)

3.2.3 Super-Resolution Transformer

The SR Transformer is encoder-only and uses the same blocks as the Hybrid Transformer encoder. The two differences are: the input has an extra binary mask channel ($D+1$) that marks which timesteps are missing, and the loss gives more weight to missing positions than observed ones.

Component	Layer	Output shape
Input projection	Dense + Pos. Embedding + Dropout	(B, W, d_{model})
	$\times N$ EncoderBlocks (as Hybrid TRN)	(B, W, d_{model})
Output	Dense	(B, W, D)

3.3 Training strategies

Important. The content of the following subsection applies only to *forecasting* models; the *super-resolution* setting does not include these training additions.

Phase	Custom options exposed in YAML
teacher_forcing	-
full_autoregressive	gradient_through_time $\in \{\text{true}, \text{false}\}$
masked_modeling	mask_prob $\in [0, 1]$; mask_scope $\in \{\text{time}, \text{element}, \text{feature}\}$; mask_mode $\in \{\text{zero}, \text{constant}, \text{noise}\}$; noise_replace $\in \{\text{true}, \text{false}\}$
scheduled_sampling	ratio_mode $\in \{\text{linear}, \text{cosine}, \text{sigmoid}, \text{power}\}$; per_feature $\in \{\text{true}, \text{false}\}$; stop_grad_pred $\in \{\text{true}, \text{false}\}$

Table 1: Compact summary of discrete customization options exposed by our training YAML.

Note. The learning rate and clipping threshold are defined globally and can be enabled for each phase, but each phase can optionally apply a per-phase scaling factor (e.g., `lr_multiplier` and `clip_norm_multiplier`) to obtain strategy-specific training dynamics: $\text{lr}_{\text{phase}} = \text{lr}_{\text{global}} \cdot \text{lr_multiplier}$ and $\text{clip}_{\text{phase}} = \text{clip}_{\text{global}} \cdot \text{clip_norm_multiplier}$.

3.3.1 Curriculum

The model is configured to predict a target sequence of length `output_seq_len`. However, during training we apply a phase-wise curriculum: instead of computing the loss over the full horizon from the beginning, at each phase k we only backpropagate the error on an effective horizon $H_k = c_k \cdot \text{output_seq_len}$, where c_k is the k -th entry of `curriculum` [1]. This allows the model to learn step by step, starting from shorter predictions and progressively reaching the full horizon.

Example. If `output_seq_len`= 50 and `curriculum`= [0.2, 1.0], then in the first phase we compute the loss only on the first $H_1 = 0.2 \cdot 50 = 10$ predicted steps, while in the second phase we use the full horizon $H_2 = 1.0 \cdot 50 = 50$.

3.3.2 Prediction mode (all vs horizon)

We also implemented `prediction_mode` to control how predictions are produced when using a curriculum. In graph mode, if the predicted length changes across epochs (because the curriculum horizon changes), the training graph may need to be rebuilt (re-tracing), which adds overhead [2]. For this reason we provide two options:

- **all**: the model always predicts `output_seq_len`, but we compute and backpropagate the loss only up to the current horizon.
- **horizon**: the model predicts only up to the horizon, and the loss is computed on the same range.

Trade-off. `all` keeps the graph stable but can be slower, since we still predict the full `output_seq_len` even for small horizons. `horizon` can be faster, but changing horizons can trigger re-tracing.

3.4 Free-running evaluation

During training, loss computed with teacher forcing or masked modeling can be too optimistic, because the model is partially guided by ground-truth inputs. To check the real inference behaviour, we added a *free-running evaluation* at specific points [3]. We ran an `inference_model` in autoregressive mode (i.e., without any help from targets) and computed losses for a part of the validation (or test) data. This helps seeing if there is exposure bias and error accumulated over time. We use three probes:

Table 2: Free-running probes used during training.

Probe	Output timesteps	When	Purpose
fr_phase	phase	every 2 epochs	Fast feedback at the current curriculum/phase horizon
fr_target	global	end of phase	Performance at the full forecasting horizon (output_seq_len)
fr_curve	{0.2,0.4,0.6}	end of phase	Error growth vs. horizon (short/medium/long range)

Note. **phase** means the current phase horizon, **global** means the global output timesteps, while for **fr_curve** we put a list of ratios compared to the global output timesteps.

Overall, **fr_phase** is useful for regular monitoring each phase, **fr_target** tracks the final goal at the desired horizon, and **fr_curve** tells us if the errors grow smoothly or explode as the prediction length increases. These signals help to see if the model is actually getting better at making predictions, not only in assisted training.

3.5 Optuna - Hyperparameter tuning

We performed hyperparameter optimization in two stages: Level-1 tunes optimizer/training and model-capacity parameters, Level-2 is initialized from the best configuration found at Level-1, taking **learning_rate** and **clip_norm** from a range of $\pm 30\%$. We didn't fix a single best Level-1 setup, because changing windowing hyperparameters (input/output sequence length and stride) can alter the optimal training and model settings. For each model we performed 20 epochs either for level 1 and 2.

We defined a scalar score to rank Optuna trials as a weighted combination of some evaluation metrics.

$$\text{Score} = 0.70 L_{\text{fr_target}} + 0.25 L_{\text{fr_curve}} + 0.05 L_{\text{fr_phase}} \quad (1)$$

Level 1

Hyperparameter	LSTM Full-seq			LSTM Step-wise			LSTM Hybrid			TRN Hybrid		
	Hp1	Hp2	Hp3	Hp1	Hp2	Hp3	Hp1	Hp2	Hp3	Hp1	Hp2	Hp3
lr	$4.6e-4$	5e-4	$4.6e-4$	$9.7e-4$	$6.7e-4$	7e-4	8.9e-4	$8.3e-4$	$6e-4$	$2.7e-4$	$6.2e-4$	6.2e-4
clip_norm	0.8699	1.4600	0.6743	0.6470	0.5780	0.8157	0.7241	0.7573	0.7361	0.1014	0.1650	0.1819
batch_size	64	64	64	32	32	32	32	32	32	32	32	32
latent_dim	256	256	256	192	192	192	192	192	192	—	—	—
num_heads	—	—	—	—	—	—	—	—	—	4	4	2
dim_model	—	—	—	—	—	—	—	—	—	256	192	128
ff_dim	—	—	—	—	—	—	—	—	—	1536	1536	1536
num_layers	—	—	—	—	—	—	—	—	—	2	2	2
dropout	—	—	—	—	—	—	—	—	—	0.1134	0.1164	0.0555
Score ↓	0.6582	0.6576	0.6578	0.2811	0.2694	0.2680	0.1719	0.1721	0.1722	0.2351	0.2103	0.1987

Table 3: Level-1 Optuna tuning. For each model we report only the best three configurations (Hp1–Hp3).

The batch size affects the noise of the gradient estimate. This, affects the step size of the optimizer at each update.

$$\text{lr}_{\text{eff}} = \text{lr}_{\text{ref}} \left(\frac{B}{B_0} \right)^\alpha, \quad B_0 = 64, \alpha = 0.5. \quad (2)$$

If we tune **learning_rate** and **batch_size** independently, some trials may become unstable (too large steps with small batches) or unnecessarily slow (too small steps with large batches). For this reason, we treat

the learning rate suggested by Optuna as a reference value and rescale it with the batch size to keep the update scale more consistent across trials, improving stability and making the comparison between configurations fairer [4].

Level 2 In Level 2 we tune three windowing hyperparameters - `input_seq_len`, `output_seq_len`, and the windowing `stride`. This leads to a bi-objective trade-off, summarized in Table 4.

Table 4: Level-2 multi-objective tuning goals.

Objective	Direction
Score	minimize ($\downarrow$)
Output length	maximize ($\uparrow$)

Since there is no single best solution, we plot the Pareto front, which collects the non-dominated trade-offs between the objectives. We then select the knee point as a good compromise [5].

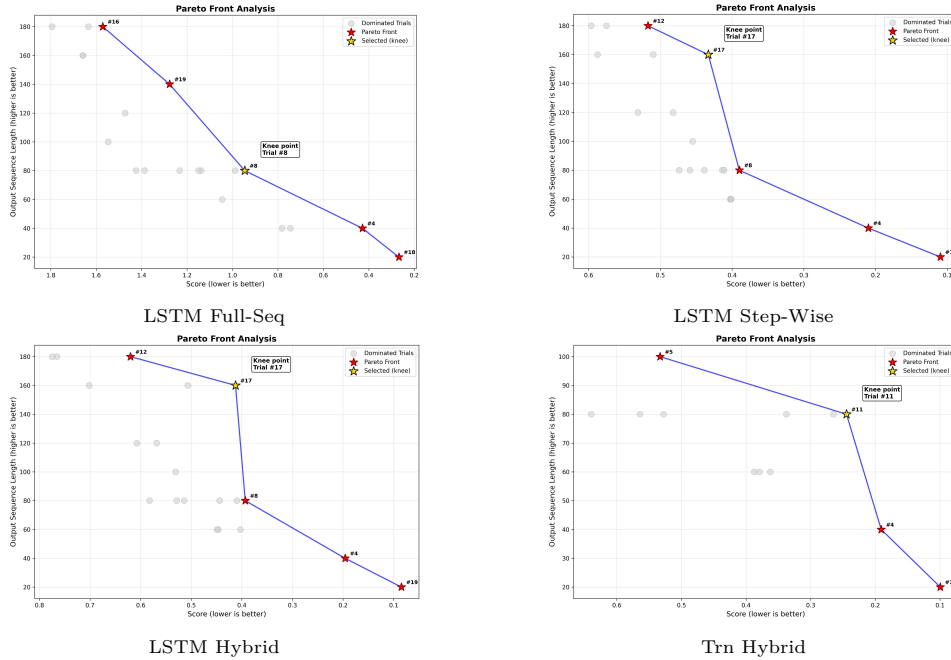


Figure 3: Pareto fronts obtained from Level-2 tuning for each model

4 Results

For each model, we select the best Level-1 configuration in Table 3. For Level 2, we select the **knee point** on the Pareto front, like in Figure 3, to obtain a good trade-off between output horizon and score. Each selected configuration is then trained for 50 epochs on an NVIDIA TESLA T4 GPU. Performance is monitored using the free-running probes described in Table 2. In the following, we show loss and prediction plots, and attention maps when available.

4.1 Forecasting

Training dynamics Training and validation losses show that the system is working as expected, while free-running metrics, using an inference model, provide a more realistic behaviour without help.

Note: each color corresponds to a different training phase with its own horizon.

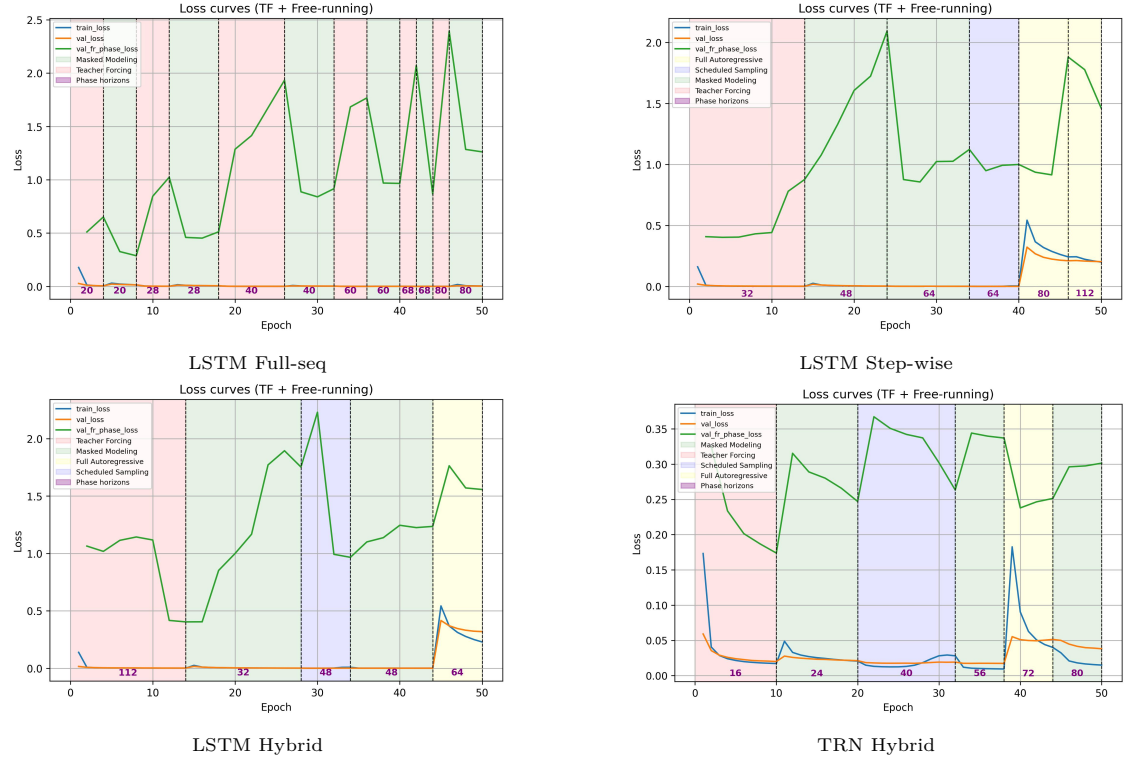


Figure 4: Forecasting training dynamics (train/val and free-running phase loss).

Train and validation loss (blue/orange) look very good most of the time, but, near the start of full autoregressive, often increase sharply, then gradually decrease again, since the model no longer receives ground truth help. Free-running loss is much higher and more unstable. This is true because, when the model must generate its own future steps, errors accumulate and the loss grows (exposure bias). When you switch the training phase, the losses often jump because the task distribution changes.

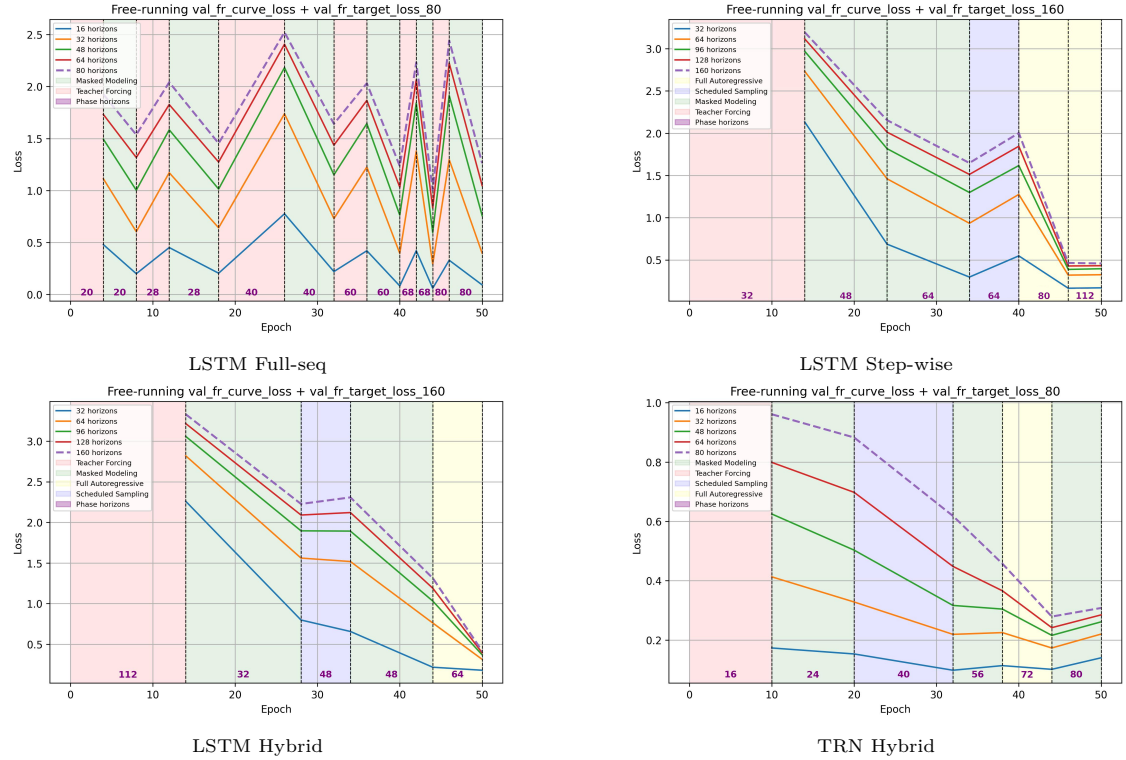


Figure 5: Forecasting training dynamics (free-running curves and target loss).

As we can see, the model is tested with different horizons. As the horizon gets closer to the target `output_seq_len`, the error usually increases, because predictions with long horizons are harder.

During teacher forcing, the free-running loss increases because when you run it autoregressively, small mistakes become the next inputs.

During scheduled sampling, some curves increase since the inputs become more self generated and noisier. When switching to full autoregressive training, losses often decrease. This behaviour is not always expected, and it can show known issues of scheduled sampling. We discuss these limitations in Future work section.

Predictions We first visualize the predictions produced by each model at its knee point configuration.

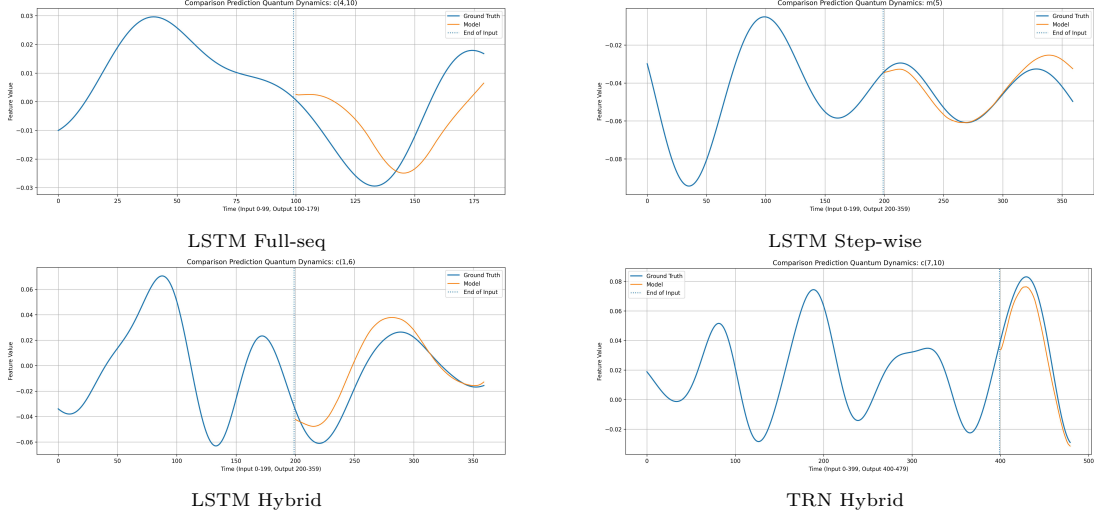


Figure 6: Predictions for each model of Knee point config

Overall, full seq decoding performs worst among the tested settings. The Transformer model is more stable at long horizons because better preserves the global structure. However, all models still show the main trend and direction of the dynamics.

These comparisons are not fully fair because each model use a different (Len_{in}, Len_{out}, s) at its knee point.

Note Len_{in} is the input sequence length, Len_{out} is the predicted sequence length, and s is the windowing stride.

Fair comparison under a fixed windowing setup. For a fairer comparison, we also evaluated all forecasting models using the same windowing configuration (Len_{in}, Len_{out}, s) .

Model	FR_target ↓	MSE ↓
LSTM Full-seq	0.7170	0.0081
LSTM Step-wise	0.2266	0.1252
LSTM Hybrid	0.2510	0.1720
TRN Hybrid	0.2444	0.0145

Table 5: Metrics comparison under a fixed windowing setup $(Len_{in}, Len_{out}, s) = (200, 50, 20)$.

Analysis. The Transformer performs strongly and is competitive with the LSTMs. As we can see in the table, the LSTM Full-Seq model has a much higher free running target loss (about 0.71) than the other models, because it was trained only with teacher forcing and masked modeling, so it didn't learn how to deal with an accumulation of errors when it was running in full autoregressive mode.

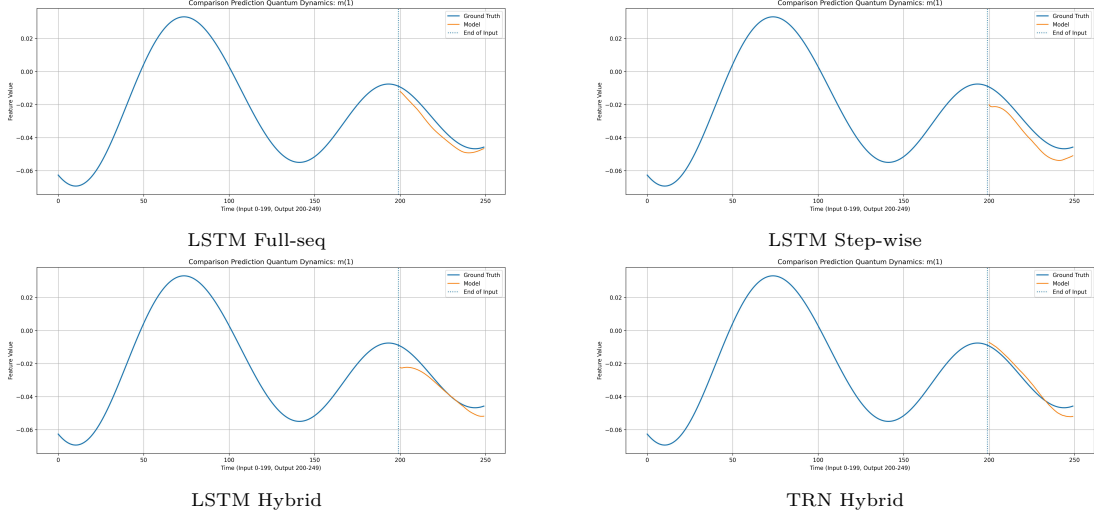


Figure 7: Predictions for each model with $(Len_{in}, Len_{out}, s) = (200, 50, 20)$.

Attention analysis (Transformer).

Note. We report attention maps only from the **decoder** of the TRN Hybrid knee-point configuration, using sample 0 and attention head 1, because they are the most interpretable for forecasting.

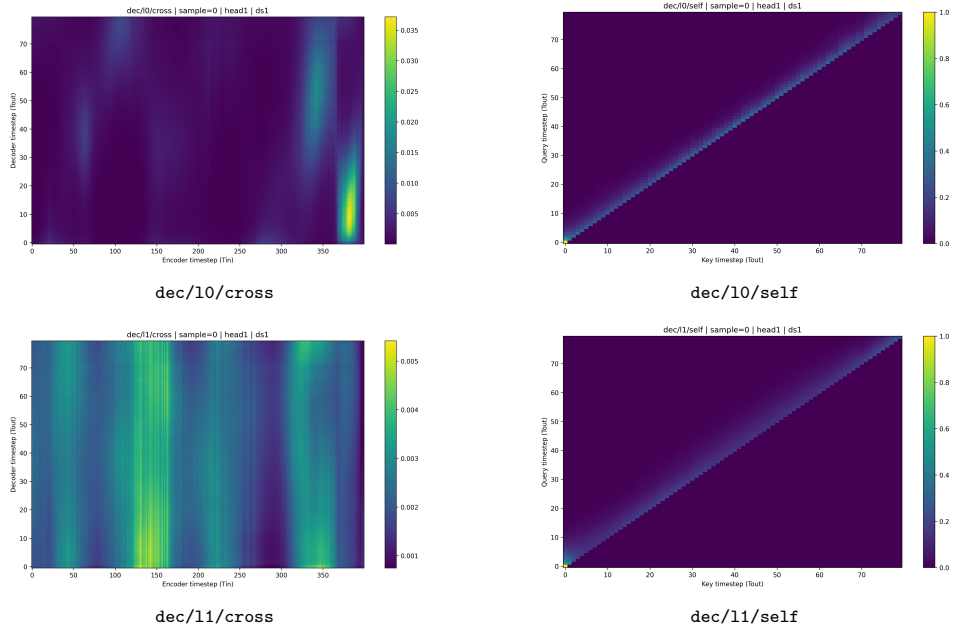


Figure 8: Attention maps for a representative sample (single head 1)

The decoder self-attention maps (**dec/10/self**, **dec/11/self**) show a clear diagonal pattern, which is expected for causal decoding. This means each predicted step mainly uses the previously generated outputs.

The cross-attention maps show that the two decoder layers work in a complementary way. The **dec/10/cross** is step-dependent: the focus over the input timesteps changes as the generated output timestep changes. We also observe a strong concentration on the last part of the input window when predicting the first output steps, which is coherent because early forecasts are strongly conditioned on the most recent observed data. The **dec/11/cross** is mostly step-invariant: it forms vertical bands, meaning that the model attends to the same input regions even when the output timestep changes. This suggests that layer 11 extracts more global information from the input that is useful across the whole forecast horizon.

4.2 Super-resolution (imputation)

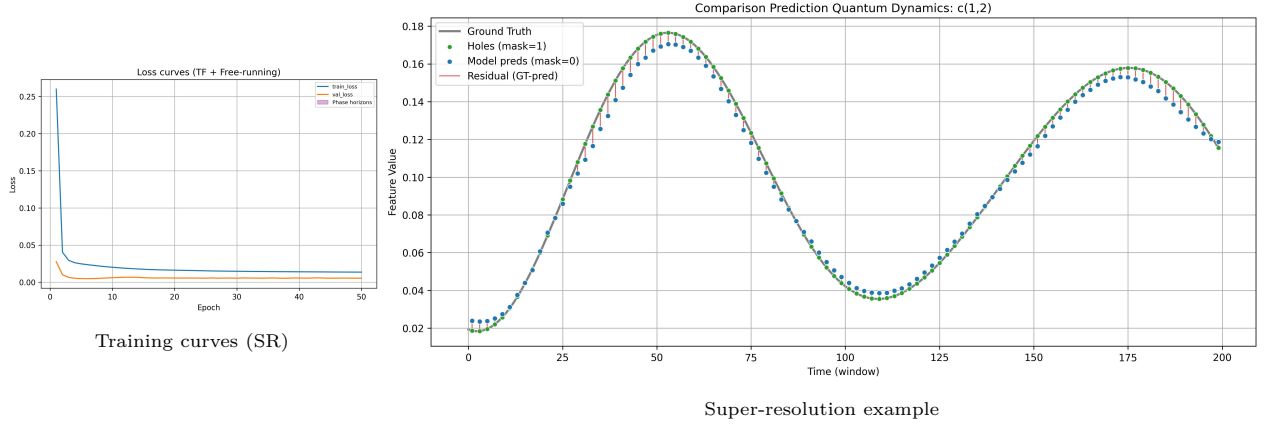


Figure 9: Super-resolution (TRN-SR): training dynamics (left) and reconstruction on a sample (right).

For super-resolution, the model performs well in general: it fills the holes while keeping the main trend. Errors tend to be higher where the dynamics change sharply.

Note. Unlike forecasting, in this super-resolution setting, we do not use the free running probes or the training strategies (e.g., scheduled sampling).

Code

All code, configurations, and scripts used for training, evaluation, and plotting are available in the project repository: `qubit-evolution-dl`.

Additional plots and detailed results (hyperparameter tuning and free running evaluation) are also included.

5 Future work

Right now, our Level-2 selection currently focuses on minimizing the score while maximizing `output_seq_len`. In future work, we can extend it by also considering `input_seq_len`. Optimizing only the score and `output_seq_len` can favor configurations that get good results simply by using a longer input window, to have more context. However, another model can reach a similar score and horizon with a shorter `input_seq_len`, which is preferable because it reduces memory and performance. So, we can think like this for the model selection: (i) minimize score, (ii) maximize `output_seq_len`, and (iii) minimize `input_seq_len`, for example, using a three-dimensional Pareto front.

Autoregressive decoding and scheduled sampling are both step-by-step: each prediction depends on the output generated at the previous step. This dependency limits parallelization and makes both training and inference slow, especially for long horizons.

Scheduled sampling has also some limitations:

- It **does not fully remove exposure bias**. This means that errors can still get worse when the model must rely on its own predictions.
- It also **creates a mismatch between train and test**, because training mixes ground truth inputs and model generated inputs in a way that is different from inference.
- In some cases it can be unstable and return suboptimal or unpredictable behaviour.

To reduce the step-wise limitation, a practical direction is to use more parallelizable variants. For Transformers, a simple option is *two-pass scheduled sampling*, which applies scheduled sampling through a two pass decoding strategy while keeping most computation parallel [6].

References

- [1] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning (ICML '09)*, 2009, pp. 41–48. DOI: 10.1145/1553374.1553380
- [2] TensorFlow Team, *Better performance with tf.function*, TensorFlow Core Guide, Accessed: 2026-02-26, 2024. [Online]. Available: <https://www.tensorflow.org/guide/function>
- [3] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, “Scheduled sampling for sequence prediction with recurrent neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. [Online]. Available: <https://arxiv.org/abs/1506.03099>
- [4] P. Goyal et al., “Accurate, large minibatch sgd: Training imagenet in 1 hour,” 2017. [Online]. Available: <https://arxiv.org/abs/1706.02677>
- [5] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002. DOI: 10.1109/4235.996017
- [6] T. Mihaylova and A. F. T. Martins, “Scheduled sampling for transformers,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019. [Online]. Available: <https://aclanthology.org/P19-2049/>